Gaussian-process reduced models versus neural regression for parametric PDEs: a staged-preregistered benchmark with largely negative confirmatory results

Authors: [AUTHOR NAMES AND AFFILIATIONS — to be supplied by the authors]. **Corresponding author:** [to be supplied].

Abstract

Background. Practitioners choose between linear-subspace (POD-based) and neural surrogates for parametric PDEs with little guidance on when either wins, and comparisons are often confounded by unequal tuning. **Methods.** We compared five surrogates (three POD-regressor and two neural) under a common random-search protocol on 36 synthetic PDE instances, with training budgets of 32–1024 solves, three seeds, tie-aware outcomes (±0.10 decade), and controls separating the subspace, the kernel prior and marginal-likelihood fitting. Hypotheses, code and success criteria of each stage were committed to version control before the corresponding runs. **Results.** Of 24 instances, the linear-subspace family was clearly better in 13, the neural family in 8 and 3 were ties at N = 1024 (at N = 32: 5, 5 and 14); on smooth three-parameter problems the advantage reached up to three decades, and held in 7 of 7 pre-selected new instances. Three of the four Stage-1 confirmatory hypotheses were not supported. A pilot-data diagnostic was not confirmed on new instances (inconclusive; it correlates with parameter dimension). The advantage was explained by marginal-likelihood hyper-parameter fitting (kernel ridge under matched search was a median 14× worse), not by the subspace (a GP without POD matched it). Longer neural training changed the class of 15 of 48 cells, nearly always toward the neural family. **Conclusions.** In this synthetic benchmark the apparent winner is governed by smoothness, dimension, regressor fitting and tuning symmetry; conclusions about ties and learning-curve crossovers are fragile. Results are limited to synthetic problems and lightly tuned networks.

Keywords: surrogate modelling; parametric PDE; reduced-order model; Gaussian process; neural network; benchmark; preregistration

1 Introduction

A parametric partial differential equation (PDE) defines a map from a vector of parameters μ to a solution field. Surrogates of this map are used in design, uncertainty quantification and control, where the full-order model (FOM) is too expensive to call many times. Practitioners typically choose between two families before spending a simulation budget: *linear-subspace* surrogates, in which proper orthogonal decomposition (POD) compresses the snapshots and a regressor maps μ to the reduced coefficients [Hesthaven & Ubbiali 2018; Guo & Hesthaven 2018; Bhattacharya et al. 2021], and *nonlinear-output* neural networks that map μ directly to the field, possibly through a latent bottleneck [Fresca & Manzoni 2022; Franco et al. 2023].

Approximation theory gives a qualitative guide: a small linear subspace suffices when the solution manifold has a rapidly decaying Kolmogorov n-width [Cohen & DeVore 2015], whereas slow decay has been established for the wave equation with discontinuous initial data [Greif & Urban 2019] and is the standard motivation for nonlinear reduced models of transport-dominated problems [Franco et al. 2023]. For operator learning with PCA-Net, slow decay of the PCA eigenvalues has been identified as an obstacle to efficient learning through lower complexity bounds [Lanthaler 2023], which motivates a spectral-decay feature as a diagnostic. It has also been argued that many neural-surrogate benchmarks sit on low-dimensional manifolds where any competent reduced model interpolates well [Duraisamy 2026, preprint]. Kernel and Gaussian-process (GP) methods have been reported to be competitive with DeepONet and the Fourier neural operator [Batlle et al. 2024]; within a POD-regression framework for extruder flow, GP regression was reported to have several advantages over a neural network [Bērziņš et al. 2020, preprint]; and a comparison of neural and polynomial operator surrogates found no universally superior method [Westermann et al. 2026, preprint]. A review of machine-learning solvers for fluid PDEs found that 79% of the articles claiming to beat a standard method used a weak baseline [McGreivy & Hakim 2024].

What remains less clear, and is the subject of this paper, is quantitative and methodological. (i) *How large* are the differences between the families, and in how many problems are they large enough to matter, as opposed to practically tied? (ii) *Why* does a linear-subspace surrogate win where it does: because of the subspace, because of the kernel prior, or because of how its hyper-parameters are fitted? (iii) Can a cheap statistic computed from a pilot set of 32 snapshots predict the winner on problems it has never seen? (iv) How sensitive is the apparent winner to the tuning protocol?

Approach and contributions. We compared five surrogates under a common random-search tuning protocol on 36 simulated parametric-PDE instances, preregistering the hypotheses, analysis code and success criteria of each stage in

version control before the corresponding results existed. Contributions: (1) a tie-aware map of where each family wins, with effect sizes, over six training budgets; (2) controlled mechanism tests that separate subspace, kernel prior, anisotropy and hyper-parameter fitting; (3) an out-of-sample test of a pilot-data diagnostic, whose frozen form was **not** confirmed; (4) a quantification of how unequal tuning inflates apparent neural-network wins, and of the limits of calling a tuning budget "matched" across method classes; (5) open code. We propose no new surrogate and claim no state-of-the-art accuracy.

2 Related work

Linear-subspace and kernel surrogates. Non-intrusive reduced-order models regress POD coefficients; neural networks were used for this by Hesthaven and Ubbiali [2018] and in PCA-Net [Bhattacharya et al. 2021], and GP regression for reduced models by Guo and Hesthaven [2018]. Bērziņš et al. [2020, preprint] compared radial-basis-function, Gaussian-process and neural-network regression inside a standardised POD framework for extruder flows and reported that GP regression offers several advantages over the neural network, which is the nearest precedent we found for our qualitative finding. Gaussian processes are the standard low-dimensional regressor [Rasmussen & Williams 2006]; a hybrid GP/neural operator-learning framework was proposed by Mora et al. [2025]. Batlle et al. [2024] develop a general kernel framework with a priori error analysis and report that it is competitive with DeepONet and the Fourier neural operator.

Neural operators and nonlinear manifolds. DeepONet [Lu et al. 2021], the Fourier neural operator [Li et al. 2021] and the neural-operator framework [Kovachki et al. 2023] learn maps between function spaces; deep autoencoders learn nonlinear latent manifolds, either inside projection-based reduced models [Lee & Carlberg 2020] or for the parameter-to-solution map [Fresca & Manzoni 2022; Franco et al. 2023]. Our setting has a finite-dimensional parameter, so we study direct and bottleneck networks rather than operator-input architectures. Slow n-width decay for transport and wave problems is established [Greif & Urban 2019].

Fair comparison. de Hoop et al. [2022] study the cost–accuracy trade-off of neural architectures; Lu et al. [2022] compare DeepONet and FNO; Westermann et al. [2026, preprint] compare neural and polynomial surrogates by Pareto frontiers over hyper-parameter ensembles, finding polynomial surrogates more data-efficient for smooth inputs and the FNO reaching the fastest convergence rates for rough ones; Zhang et al. [2026, preprint] define a breakeven complexity that includes data-generation and training costs, while Huang and Stuart [2026, preprint] study post-training many-query cost against classical solvers with data and training costs treated as amortised; McGreivy and Hakim [2024] document weak baselines and reporting bias; Grossmann et al. [2024] compare physics-informed networks with finite elements. Earlier work attempted to derive rules for choosing a surrogate form from data characteristics for design-space approximation [Williams & Cremaschi 2019]. Random search is an efficient alternative to grid search for hyper-parameter optimisation [Bergstra & Bengio 2012], and reporting results as a function of tuning budget and accounting for run-to-run variance have been advocated [Dodge et al. 2019; Bouthillier et al. 2021].

Positioning. The qualitative regime picture, the competitiveness of kernel/GP methods and a preference for GP over neural regression in POD frameworks are established. The present work adds a matched-protocol quantification across the instance suite with equivalence bands, mechanism controls for the Gaussian-process advantage, a pre-registered out-of-sample test of a pilot diagnostic, and a measurement of tuning-protocol sensitivity. *Reading depth and search coverage.* Reading was limited and uneven. Westermann et al. and Zhang et al. were read in large parts of a converted HTML text (we cannot give a reliable percentage); all other sources were assessed from abstracts or bibliographic metadata, a few from titles only. The searches covered the arXiv API, Crossref and web search; one targeted arXiv query for POD+GP versus neural networks returned no hits; Google Scholar, publisher full texts and patent databases were not searched. The absence of a prior pilot-data selection rule is therefore weak evidence.

3 Surrogate families, tuning protocol and diagnostics

Problem. A parametric PDE defines a map $\mu \in \mathbb{R}^p \mapsto u(\mu) \in \mathbb{R}^m$, where u is the FOM solution on an output grid ($m \in \{128, 256\}$). Given N training pairs, a surrogate $\hat{u}$ is scored by the mean relative L_2 error $\mathrm{err} = (1/n)\, \Sigma_j \, \|\hat{u}(\mu_j) - u(\mu_j)\|_2 \,/\, \|u(\mu_j)\|_2$ over $n = 1000$ held-out parameters.

Linear-subspace (LS) family. POD of the centred training snapshots gives $u(\mu) \approx \bar{u} + V_r\, a(\mu)$; the methods differ in the regressor for $a(\mu)$: *POD+GP* (ARD Gaussian processes with Matérn-5/2 or RBF kernel, hyper-parameters fitted by maximising the marginal likelihood on up to 300 points for the leading four modes, the remaining modes reusing the fourth kernel); *POD+Poly* (ridge-regularised total-degree polynomial, degree ≤ 6, ≤ 5,000 features); *POD+MLP* (multilayer perceptron on the scaled coefficients).

Nonlinear-output (NL) family. *MLP (direct)* maps μ to the full field; *Bottleneck MLP* passes μ through a latent layer of width $k \in \{2, 4, 8, 16\}$ before the field decoder.

Mechanism controls. *POD+KRR*: POD with kernel ridge regression (Matérn-5/2 or RBF, isotropic length-scale, ridge), tuned by the same random search as every other method and **without** marginal-likelihood fitting. *POD+GP-iso*: POD with a GP whose kernel has a single isotropic length-scale, fitted by marginal likelihood. Together with POD+GP (anisotropic, marginal-likelihood fitted), these separate the roles of the kernel prior, anisotropy and marginal-likelihood fitting.

Tuning protocol. Every method receives exactly $B = 8$ random-search trials (trial 0 is a fixed default) scored on a random 20% validation split of the N samples; the validation-best configuration is refit on all N samples. Neural networks use Adam with cosine learning-rate decay for 1500 epochs (float32, CPU). Test errors of each trial are logged for the tuning analysis but never used for selection.

Outcome. For each (instance, N, seed), LS* and NL* are the validation-best methods of each family; $\bar{y}$ = mean over seeds of $\log_{10}$(err_NL* / err_LS)*, so $\bar{y} > 0$ means the LS family is better. A cell is a* tie* if $|\bar{y}| \le 0.10$ decades (an error ratio within about 26%), *LS-clear* if $\bar{y} > 0.10$ and *NL-clear* if $\bar{y} < -0.10$.

Pilot diagnostic. From the first $n_0 = 32$ training snapshots only, f1 = $\log_{10}$ of the mean leave-one-out relative POD reconstruction error with $r = 4$ modes, and f2 = the slope of $\log_{10} \sigma_k$ against k ($k \le 10$).

4 Experimental design and preregistration

Instances. Thirty-six simulated instances in five sets (Table 1): the *original suite* (16 instances, 8 families × 2 levels), *Experiment A* (8 new instances in 4 new families: high-dimensional Darcy with p = 16 and 32, a Darcy problem with a moving discontinuous inclusion, KdV solitons, and smooth nonlinear conduction), *Experiment A2* (8 further candidates: nonlinear reaction–diffusion, Helmholtz-type, heat with parametrised diffusivity, 1-D diffusion) and *Experiment A3* (4 probes of the diagnostic–dimension confound: two nearly linear Darcy problems with p=16 and 32, and two p=3 problems with moving features). Each solver is verified by unit tests against exact or manufactured solutions (Poisson series, translation, d'Alembert, KdV soliton, Helmholtz and reaction–diffusion manufactured solutions) or grid refinement. **All data are synthetic.**

Data and budgets. For every instance: 4096 i.i.d. training parameters, 1000 interpolation-test and 500 extrapolation-test parameters, from streams seeded by the instance name. Seed s uses the first N elements of a seed-specific permutation, $N \in \{32, 64, 128, 256, 512, 1024\}$ (A2 and the isotropic-GP control: $N \in \{64, 256, 1024\}$), seeds 0–2.

Staged preregistration (all in version control; commit hashes in the supplement). *Stage 1* (original study): hypotheses H1a, H1b, H2, H4a, analysis code and success criteria committed before any result. *Stage 2*: after a forensic audit of Stage 1, a revised protocol was committed before any Stage-2 comparative result: Experiment A (frozen rules derived from Stage-1 data only; primary tests P1, P2, secondary S1), Experiment B (stronger neural-network refits with explicit sign-retention criteria), Experiment D (kernel-ridge mechanism test with a pre-specified criterion). *Stage 3*: after seeing Experiment A (diagnostic not confirmed) and Experiment D, an addendum fixed, before the runs, Experiment A2 (the 8 candidates; qualifying instances chosen by a pilot-only rule f1 < −2; a prediction and its success criteria) and Experiment E (isotropic-GP control with a pre-specified decision rule). Stage-3 experiments are post hoc in *motivation* but preregistered in *prediction*. The preregistrations are git histories on the authors' repository, not independent registry entries.

Statistics. Stage-1 confirmatory tests (H1a, H1b, H2, H4a) used Holm correction; Experiment A tests P1 and P2 used Holm correction; all other analyses are labelled exploratory. The unit of inference is the instance (or family), not the (instance, N) cell.

Table 1 / per-instance outcomes at N = 1024

Instance	p	f1	y1024	Class	Set
bratu_L1	3	-3.75	3.31	LS-clear	original
bratu_L2	3	-3.27	3.08	LS-clear	original
helm_L1	3	-3.94	2.93	LS-clear	A2
heatp_L2	3	-3.37	2.9	LS-clear	A2
heatp_L1	3	-3.79	2.87	LS-clear	A2
nlheat_L1	3	-3.91	2.72	LS-clear	Exp A
diff1d_L1	4	-2.18	2.7	LS-clear	A2
helm_L2	3	-4.16	2.64	LS-clear	A2

Instance	p	f1	y1024	Class	Set
nlheat_L2	3	-3.45	2.58	LS-clear	Exp A
reacdiff_L1	3	-4.13	2.57	LS-clear	A2
reacdiff_L2	3	-3.75	2.38	LS-clear	A2
burgers_L1	4	-2	2.25	LS-clear	original
heat2d_L1	4	-1.15	1.38	LS-clear	original
darcy_L1	4	-1.16	1.13	LS-clear	original
diffld_L2	8	-1.72	1	LS-clear	A2
wave_L1	4	-0.62	0.78	LS-clear	original
transport_L1	4	-0.88	0.75	LS-clear	original
fisher_L1	4	-1.81	0.52	LS-clear	original
darcylin_L1	16	-1.57	0.48	LS-clear	A3
kdv_L2	4	-1.03	0.29	LS-clear	Exp A
advdiff_L1	4	-0.78	0.21	LS-clear	original
darcylin_L2	32	-1.6	0.18	LS-clear	A3
burgers_L2	4	-0.65	0.11	LS-clear	original
wave_L2	4	-0.27	-0.04	tie	original
darcyhd_L2	32	-0.49	-0.07	tie	Exp A
advdiff_L2	4	-0.59	-0.07	tie	original
darcyhd_L1	16	-0.49	-0.13	NL-clear	Exp A
incl_L2	4	-1.29	-0.13	NL-clear	Exp A
darcy_L2	10	-0.36	-0.19	NL-clear	original
incl3_L1	3	-1.29	-0.21	NL-clear	A3
kdv_L1	4	-0.57	-0.23	NL-clear	Exp A
incl_L1	4	-1.69	-0.25	NL-clear	Exp A
transport_L2	4	-0.42	-0.33	NL-clear	original
heat2d_L2	4	-0.22	-0.34	NL-clear	original
fisher_L2	4	-1.61	-0.44	NL-clear	original
heatmv3_L1	3	-0.23	-0.44	NL-clear	A3

Table 1. Instances ordered by $\bar{y}_{1024}$ (> 0: linear-subspace better). f1 is the pilot diagnostic; p the parameter dimension. Sets: original (16), Exp A (8, new families, independent test of a frozen rule), A2 (8 smooth candidates), A3 (4 probes).

5 Results

All numbers below are produced by `manuscript/build_v3.py` from the result files in the repository; the verdicts are against criteria registered before the corresponding runs (§4). Unless stated, $\bar{y}$ is the seed-mean of $\log_{10}(\text{err_NL}/\text{err_LS})$ and the unit of inference is the instance.

5.1 Where each family wins

Across the 24 instances of the original study and Experiment A, the outcome depends strongly on the training budget (Figure 1, Table 2). At N = 32, 5 instances favour the linear-subspace family clearly, 5 favour the neural family clearly and 14 are ties; at N = 1024 the counts are 13, 8 and 3. Where the linear-subspace family wins, it can win by orders of magnitude: $\bar{y}_{1024}$ = 3.3 (bratu_L1), 2.7 (nlheat_L1), 2.2 (burgers_L1); where the neural family wins, the margin is at most 0.44 decade. Of the 8 further smooth candidates of Experiment A2, 8 of 8 are linear-subspace-clear at N = 1024 ($\bar{y}$ between 1.0 and 2.9). The neural-network-favourable cases are concentrated in problems with moving or discontinuous features (transport_L2, heat2d_L2, incl, kdv_L1) and in some high-dimensional parameter spaces (darcyhd, p = 16 and 32). The earlier headline that neural networks “win most cells at N = 32” is not supported: most of those cells are ties (Figure 1).

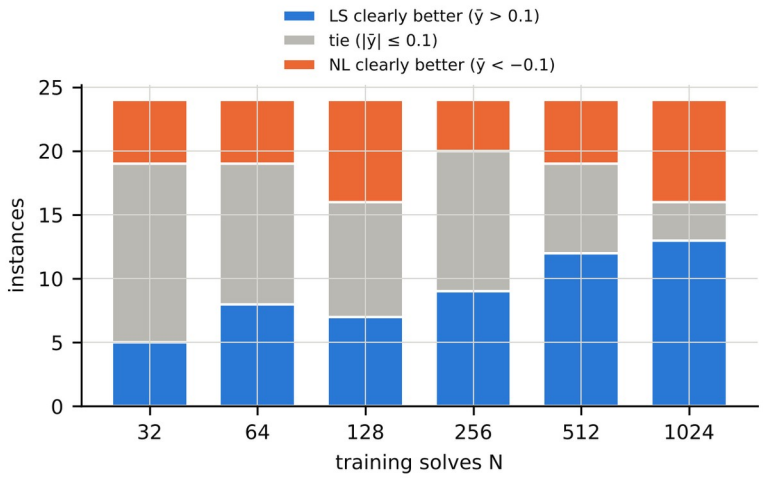


Figure 1. Number of instances (out of 24: 16 original + 8 Exp A) in which the linear-subspace family is clearly better, tied, or clearly worse than the best neural network at each training budget (tie band ±0.10 decade).

N	LS-clear	tie	NL-clear
32	5	14	5
64	8	11	5
128	7	9	8
256	9	11	4
512	12	7	5
1024	13	3	8

Table 2. Cell classification of the 24 instances by training budget (tie band ±0.10 decade).

Stage-1 confirmatory hypotheses were largely not supported (Table 3): the family difference at N = 32 has median $\bar{y}$ = -0.059, and $\bar{y}$ did not fall systematically with N (5 of 16 instances). **The pre-registered trend of $\bar{y}$ with N must be read with a confound in mind**: the training loop uses full-batch Adam (1500 steps) for N ≤ 512 but mini-batches of 512 for N = 1024 (3000 steps), so the neural family receives twice the optimisation steps at the largest budget, and its error improves visibly more from N = 512 to 1024 than from 256 to 512 (design audit; step-count note in the repository). We therefore make no claim about learning-curve crossovers.

ID	Hypothesis / criterion	Observed	raw p	Holm-adj. p	Verdict
H1a (Stage 1)	median $\bar{y}$ at N=32 > 0	median -0.059	0.874	1.000	not supported
H1b (Stage 1)	$\bar{y}$ falls with N in most instances	5/16 with ρ<0	0.962	1.000	not supported
H2 (Stage 1)	pilot features beat {$\log_2$N, p} (leave-one-family-out)	ΔRMSE 3.25, CI [-0.07, 6.40]	0.048	0.145	not supported
H4a (Stage 1)	asymmetric tuning inflates NL wins	10 vs 1 discordant cells; NL-win share 47%−56%	0.006	0.023	supported at cell level; see text
P1 (Exp A)	Spearman(f1, $\bar{y}_{1024}$) < 0, 8 new instances	ρ = -0.381	0.18	0.180	not rejected
P2 (Exp A)	frozen R2 beats frozen B0 out of sample	gain 0.598, CI [-0.001, 1.33] (10^6 draws)	0.025	0.051	not rejected (knife-edge)
S1 (Exp A)	frozen threshold rule beats majority (70.8%)	6/10 correct	0.864	–	not supported

Table 3. Pre-registered confirmatory and primary tests. Holm adjustment within each stage. "Knife-edge": the confidence interval's lower limit is −0.001.

5.2 A pilot diagnostic: inconclusive

A frozen rule built from the original data (threshold t = −0.52 on f1; a two-feature regression R2) was tested out of sample on 8 new instances (4 new families, two levels each). P1 was not significant (ρ = -0.381, exact p = 0.180) and P2 was a near miss (out-of-sample squared-error gain 0.598; instance-bootstrap p = 0.025, Holm-adjusted 0.051); the

frozen threshold rule classified 6 of 10 non-tie cells correctly, below the 70.8% majority baseline. The pre-registered verdict is therefore *not replicated*, but the right reading is *inconclusive*: the test has 8 instances in 4 families, its power at the pre-specified effect is modest (≈0.5 in a simulation by an independent audit), 88% of the P2 gain comes from the two nlheat instances, and the correlation changes sign without them ($\rho = 0.49$). In addition, f1 is correlated with the parameter dimension (Spearman $\rho = 0.71$ across the 24 instances) and every instance with f1 < −2 has p = 3 (Figure 2).

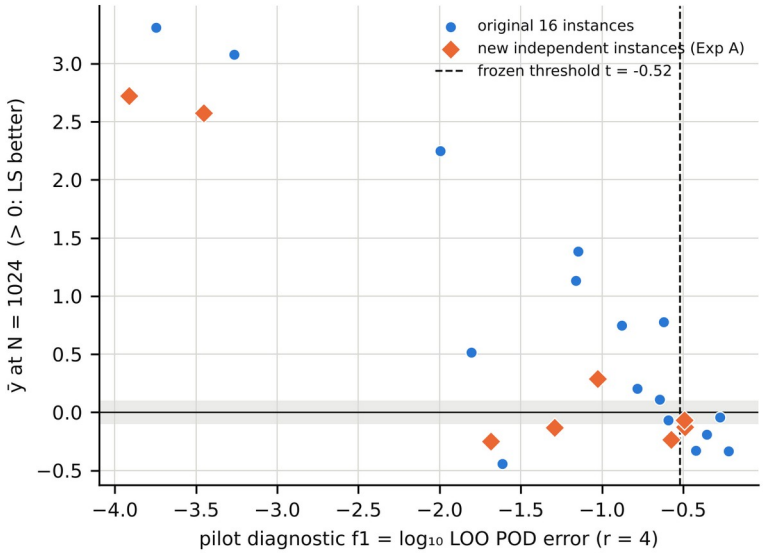


Figure 2. Pilot diagnostic f1 against the outcome $\bar{y}$ at N=1024 for the 16 original (blue) and 8 independent Exp A (orange) instances; the dashed line is the threshold frozen from the original data. Instances with f1<−2 all have p=3.

Two follow-up experiments probed this confound (neither can be confirmatory). In Experiment A2, a pilot-only rule (f1 < −2) selected 7 of 8 candidate instances; all 7 reached $\bar{y}_{1024} \geq 1$ decade (pre-registered criterion: ≥ 4 of 7 against a baseline rate of 0.15; exact p = 1.7e-06) and none favoured the neural family. But six of the seven have p = 3, and the single control (p = 8, f1 = −1.72) also had $\bar{y}_{1024} = 1.00$. In Experiment A3, two p = 3 problems with moving features (f1 = -0.23, -1.29) favoured the neural family ($\bar{y}_{1024}$ = -0.44, -0.21), indicating that f1 carries information beyond dimension, while two nearly linear high-dimensional problems (p = 16, 32) had f1 ≈ −1.6, by construction (r = 4 modes cannot capture a nearly linear map in 16 or 32 parameters), and only mildly favoured the linear-subspace family ($\bar{y}_{1024}$ = 0.48, 0.18). Pooled over the 24 instances and leaving one family out at a time, adding f1 to $\{\log_2 N\}$ lowers the cross-validated RMSE from 1.07 to 0.62 (exploratory; the features were first selected on 16 of these instances, so this is not independent evidence). Our summary is that a cheap pilot statistic is informative about *where the linear-subspace family wins by a lot* (smooth, low-dimensional maps), but we cannot show that it is more than a smoothness-and-dimension indicator.

5.3 Why the linear-subspace family wins: the regressor and its fitting, not the subspace

The kernel-ridge control, tuned by the same B = 8 random search as every other method but without marginal-likelihood fitting, is much worse than POD+GP: in the 52 cells where POD+GP clearly beats the best network, the median error ratio is 14.3× (only 13% of cells within 3×; 90–490× in the smoothest instances), so the pre-registered conjecture that matched tuning would close the gap (D1) is rejected. An isotropic-kernel GP fitted by marginal likelihood is within a median 1.92× of the anisotropic GP, whereas kernel ridge is 8.1× worse than the isotropic GP; by the pre-registered rule this identifies marginal-likelihood fitting (**M-MLII**) rather than anisotropy as the main source of the advantage in the cells where it exists (p = 3 and 4 only; the experiment says nothing about high-dimensional parameter spaces). A GP on the full field without POD matches POD+GP (median ratio 0.95×; G1 supported), so subspace truncation is not what makes the family accurate. Finally, a *direct* neural network beats the neural network on POD coefficients by more than 0.1 decade in 17 of 24 instances at N = 1024 (the opposite in 1), whereas POD+GP beats the best direct/bottleneck network by more than 0.1 decade in 12. The family label “linear-subspace” therefore mostly stands for “Gaussian-process regression with fitted hyper-parameters” in this benchmark.

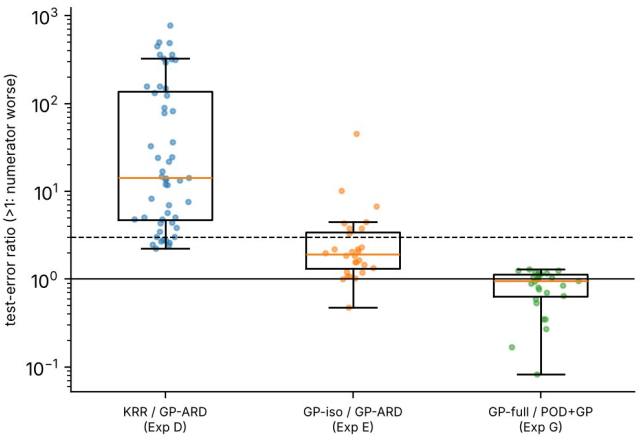


Figure 3. Mechanism controls in cells where POD+GP-ARD clearly beats the best neural network. Dashed line: 3× (the pre-registered tolerance). Left: matched-search kernel ridge (no marginal-likelihood fitting); middle: isotropic-kernel GP; right: GP on the full field without POD.

Experiment	Quantity	Observed	Pre-registered verdict
D (KRR control)	median err(POD+KRR)/err(POD+GP) in GP-clear cells	14.3× (share ≤3×: 13%; n=52)	D1 (≤3×, ≥75%) **rejected**
E (iso-GP control)	median err(GP-iso)/err(GP-ARD); err(KRR)/err(GP-iso)	1.92×; 8.1× (n=28)	**M-MLII** (low-p cells only)
G (no-POD GP)	median err(GP-full)/err(POD+GP) in GP-clear cells	0.95× (q25–q75 0.64–1.14; all ≤3×)	G1 **supported**

Table 4. Mechanism experiments (all post hoc in motivation; criteria registered before the runs).

5.4 Robustness, and the limits of "matched tuning"

Check	Result
Tie band 0.05 / 0.10 / 0.15 / 0.30, N=1024 (LS-clear / NL-clear / tie)	13/10/1 ; 13/8/3 ; 12/6/6 ; 10/3/11
Tie band, N=32 (same order)	5/10/9 ; 5/5/14 ; 4/4/16 ; 2/0/22
Seed sign-consistency of clear cells	mean share of seeds with the cell's sign 95.5%; all 3 seeds agree in 87% of 89 clear cells
Config-draw halves (even vs odd random-search trials)	class agreement 83% over 144 cells; sign agreement 99% when both halves clear; Pearson r($\bar{y}$) = 0.988
Metric: median or 95th-percentile error instead of mean	class agreement 95% / 91%; sign flips among 89 clear cells: 0 / 0
NN search-space boundary (selected configs at an edge)	73% of 1296 selected configs; 72% in LS-clear cells vs 74% elsewhere
Exp B: 4× epochs (6000 vs 1500), N∈{256,1024}	B1 88% (≥85%) pass; B2 median shift 0.24 (<0.5) pass; but 15/48 cell classes change

Table 5. Robustness checks. The first six rows are analysis-only; the Exp B row refits stored configurations.

Two results deserve emphasis. First, the class of a cell is only moderately reproducible under a different random-search draw: even and odd search trials give the same class in 83% of cells, though the sign of clear cells agrees in 99%; conclusions about individual *ties* are fragile, conclusions about large gaps are not. Second, **the neural error floor is partly a budget effect**. Training four times longer (Exp B) satisfies both pre-registered criteria (sign retained in 88% of clear cells; median shift toward the neural family 0.24 decade), but it moves $\bar{y}$ systematically toward the neural family, changes the class of 15 of 48 cells (nine ties become neural-clear), and improves 90% of the refits. The headline of large linear-subspace wins in smooth cells is robust to this (bratu remains ≈ 2.6–2.9 decades), whereas the tie/neural-win boundary is not. **Exp F (float64 + L-BFGS; pre-registered artefact test).** At N=1024 in the 7 instances where the linear-subspace family led by ≥1 decade, the best of the 1500-epoch float32, 6000-step float64 Adam and float64 Adam+L-BFGS refits lowers the neural error floor by a median 0.47 decade; median $\bar{y}$ moves from 2.51 to 2.04; 5 of 7 instances keep $\bar{y} \geq 1$. Pre-registered verdict: **ROBUST**. The instances that fall below one decade are darcy_L1, heat2d_L1, none reverses to a neural win. The oracle-best variant is chosen using test error, which favours the neural family by design. The smallest neural error reached falls from 6.8e-04 to 2.0e-04.

5.5 Sensitivity to tuning symmetry

In the Stage-1 test of unequal tuning, giving the neural family 8 trials and the linear-subspace family 1 changed the winner in 10 cells in the direction of the neural family and in 1 in the other (exact McNemar p = 0.006; neural-win share 47% → 56%). This is a cell-level test; at the instance level only 5 of 16 instances contribute any flip (Wilcoxon p = 0.031). We read it as direction-consistent evidence that asymmetric tuning favours whichever side is tuned more,

not as a precise effect size. "B = 8 trials" counts trials, not coverage of the respective search spaces: the POD+GP search has only 8 discrete points and its marginal-likelihood optimisation is not counted in B, whereas the neural search is continuous in several dimensions; and 73% of the selected neural configurations lie at an edge of their search space.

6 Discussion

What the evidence supports. (i) In this suite of 36 synthetic instances, the best Gaussian-process reduced model is far more accurate than float32 networks trained for 1500 epochs on smooth, low-dimensional maps, at training budgets of 256–1024 solves, and the advantage is reproduced in 7 of 7 pre-selected new smooth instances. (ii) The advantage is largely due to marginal-likelihood fitting of the regressor, and does not need the POD basis. (iii) Neural networks are competitive or better on problems with moving or discontinuous features and in some high-dimensional parameter spaces, and at small N most cells are ties. (iv) The classification of a cell is sensitive to the neural training budget and, for ties, to the random-search draw.

What it does not support. We could not confirm that a pilot-data statistic predicts the winner on unseen instances (inconclusive at 8 instances), nor that learning curves cross as data grow (confounded with step count). We make no claim about real-world application problems, about operator-learning architectures (FNO, DeepONet), about tuned optimisers beyond the checks reported, or about problems with $p \gtrsim 10$ where Exp E is uninformative.

Relation to prior work. The competitiveness of kernel/GP methods [Batlle et al. 2024], the advantage of GP regression over a neural network in a POD framework [Bērziņš et al. 2020] and the absence of a universal winner [Westermann et al. 2026] are known. Our contribution is a quantified, tie-aware map under a common protocol, controls that attribute the GP advantage to hyper-parameter fitting rather than to the subspace or anisotropy, and an explicit report of how far the answer moves with tuning symmetry, training budget and the draw of random-search configurations. We do not claim to be the first to observe that GP beats a network on smooth problems.

7 Limitations

Synthetic problems only; 36 instances in 15 families, several of them small (1-D/2-D, 128–256 outputs), solved by in-house solvers verified by manufactured solutions but not by a third-party code. *Two families of methods only*, five main methods plus three controls; no operator-learning networks, no ensembles, no second-order optimisers in the main protocol. *Neural networks are lightly tuned*: B = 8 random draws, a restricted search space with many selected configurations at its edge, Adam for 1500 epochs; Exp B and Exp F show that the neural error moves by a few tenths of a decade (Exp B) with larger budgets; a thoroughly tuned neural pipeline could narrow some gaps. *Tuning is "matched" only in the number of trials. The diagnostic is confounded with parameter dimension and carried by a few smooth instances. Staged preregistration is self-hosted* (git history on the authors' repository, with commit hashes recorded; not an independent registry), several experiments were designed after seeing earlier results (post hoc in motivation), and the original run had two disclosed deviations (a non-deterministic seeding of the random search, fixed and replay-verified; an interim look at two of three seeds). *The statistical units are few*: 24 instances in 12 families; cell-level tests are not independent. *Compute*: CPU, float32 unless stated; timings are not reported as a claim.

8 Conclusions

On 36 synthetic parametric-PDE instances, Gaussian-process reduced surrogates beat lightly tuned neural networks by up to three orders of magnitude on smooth, low-dimensional maps, the advantage is attributable to marginal-likelihood fitting rather than to the linear subspace, and neural networks are at least competitive on problems with moving features and in several high-dimensional cases. A pilot-data diagnostic for predicting the winner was not confirmed on new instances. Practitioners comparing surrogate families should report tie-aware outcomes, state the tuning symmetry and training budget, and include a hyper-parameter-fitted GP baseline.

Declarations

Author contributions, affiliations, funding, competing interests: [to be supplied by the authors]. *Use of AI tools:* the study design, code, analyses and this manuscript were prepared with an AI system (Claude) acting as research assistant under the authors' direction; the "independent" audits in the supplement were performed by fresh-context AI agents, not by human reviewers, and the authors must verify every number, citation and claim before submission and take responsibility for it. *Data and code availability:* all data are generated by the code in the repository (`Reinforced_Research_Project/code`) from the instance names and seeds; raw outputs are in `results/`; a permanent archive (e.g. Zenodo DOI) has not been created yet. *Preregistration:* staged, in git history (commit hashes in `research_design/`), not in an independent registry.

References

[1] Batlle P, Darcy M, Hosseini B, Owhadi H (2024) Kernel methods are competitive for operator learning. J Comput Phys 496:112549. 10.1016/j.jcp.2023.112549 / arXiv:2304.13202

[2] Bergstra J, Bengio Y (2012) Random search for hyper-parameter optimization. J Mach Learn Res 13(10):281-305. JMLR 13:281-305

[3] Bhattacharya K, Hosseini B, Kovachki NB, Stuart AM (2021) Model reduction and neural networks for parametric PDEs. SMAI J Comput Math 7:121-157. 10.5802/smai-jcm.74

[4] Bouthillier X, Delaunay P, Bronzi M, et al. (2021) Accounting for variance in machine learning benchmarks. MLSys 2021. arXiv:2103.03098

[5] Bērziņš A, Helmig J, Key F, Elgeti S (2020) Standardized non-intrusive reduced order modeling using different regression models with application to complex flow problems. arXiv:2006.13706.

[6] Cohen A, DeVore R (2015) Approximation of high-dimensional parametric PDEs. Acta Numerica 24:1-159. 10.1017/s0962492915000033

[7] Dodge J, Gururangan S, Card D, Schwartz R, Smith NA (2019) Show your work: improved reporting of experimental results. EMNLP-IJCNLP 2019:2185-2194. 10.18653/v1/d19-1224

[8] Duraisamy K (2026) Predictivity and utility of neural surrogates of multiscale PDEs. arXiv:2604.20061.

[9] Franco NR, Manzoni A, Zunino P (2023) A deep learning approach to reduced order modelling of parameter dependent partial differential equations. Math Comp 92(340):483-524. 10.1090/mcom/3781

[10] Fresca S, Manzoni A (2022) POD-DL-ROM: enhancing deep learning-based reduced order models for nonlinear parametrized PDEs by proper orthogonal decomposition. Comput Methods Appl Mech Eng 388:114181. 10.1016/j.cma.2021.114181

[11] Greif C, Urban K (2019) Decay of the Kolmogorov N-width for wave problems. Appl Math Lett 96:216-222. 10.1016/j.aml.2019.05.013

[12] Grossmann TG, Komorowska UJ, Latz J, Schönlieb C-B (2024) Can physics-informed neural networks beat the finite element method? IMA J Appl Math 89(1):143-174. 10.1093/imamat/hxae011

[13] Guo M, Hesthaven JS (2018) Reduced order modeling for nonlinear structural analysis using Gaussian process regression. Comput Methods Appl Mech Eng 341:807-826. 10.1016/j.cma.2018.07.017

[14] Hesthaven JS, Ubbiali S (2018) Non-intrusive reduced order modeling of nonlinear problems using neural networks. J Comput Phys 363:55-78. 10.1016/j.jcp.2018.02.037

[15] Huang DZ, Stuart AM (2026) Cost-accuracy trade-offs: neural operator vs classical numerical solver. arXiv:2609.24021.

[16] Kovachki N, Li Z, Liu B, Azizzadenesheli K, Bhattacharya K, Stuart A, Anandkumar A (2023) Neural operator: learning maps between function spaces with applications to PDEs. J Mach Learn Res 24(89):1-97. JMLR 24(89)

[17] Lanthaler S (2023) Operator learning with PCA-Net: upper and lower complexity bounds. arXiv:2303.16317.

[18] Lee K, Carlberg KT (2020) Model reduction of dynamical systems on nonlinear manifolds using deep convolutional autoencoders. J Comput Phys 404:108973. 10.1016/j.jcp.2019.108973

[19] Li Z, Kovachki N, Azizzadenesheli K, et al. (2021) Fourier neural operator for parametric partial differential equations. ICLR 2021. arXiv:2010.08895

[20] Lu L, Jin P, Pang G, Zhang Z, Karniadakis GE (2021) Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators. Nat Mach Intell 3:218-229. 10.1038/s42256-021-00302-5

[21] Lu L, Meng X, Cai S, et al. (2022) A comprehensive and fair comparison of two neural operators (with practical extensions) based on FAIR data. Comput Methods Appl Mech Eng 393:114778. 10.1016/j.cma.2022.114778

[22] McGreivy N, Hakim A (2024) Weak baselines and reporting biases lead to overoptimism in machine learning for fluid-related partial differential equations. Nat Mach Intell 6(10):1256-1269. 10.1038/s42256-024-00897-5

[23] Mora C, Yousefpour A, Hosseinmardi S, Owhadi H, Bostanabad R (2025) Operator learning with Gaussian processes. Comput Methods Appl Mech Eng 434:117581. 10.1016/j.cma.2024.117581

[24] Rasmussen CE, Williams CKI (2006) Gaussian processes for machine learning. MIT Press. 10.7551/mitpress/3206.001.0001

[25] Westermann J, Huber B, O'Leary-Roseberry T (2026) Performance of neural and polynomial operator surrogates. arXiv:2604.00689.

[26] Williams B, Cremaschi S (2019) Surrogate model selection for design space approximation and surrogate-based optimization. Comput Aided Chem Eng 47:353-359. 10.1016/B978-0-12-818597-1.50056-4

[27] Zhang Y, Roberts N, Marwah T (2026) Breakeven complexity: a new perspective on neural partial differential equation solvers. arXiv:2605.15399.

[28] de Hoop MV, Huang DZ, Qian E, Stuart AM (2022) The cost-accuracy trade-off in operator learning with neural networks. J Mach Learn 1(3):299-341. 10.4208/jml.220509